

Activation Functions Explained

ReLU, Sigmoid, Tanh & Softmax — with real dataset examples

1. ReLU (Rectified Linear Unit)

Formula: $f(z) = \max(0, z)$

- If $z > 0 \rightarrow \text{output} = z$
- If $z \leq 0 \rightarrow \text{output} = 0$

Example Dataset: House Price Prediction

Predicting house price contribution based on **size (sq ft)**.

Size (sq ft)	Weighted Sum (z)	ReLU Output
1200	300	300
800	200	200
400	-50	0
1500	400	400

Step-by-Step

1. Input Feature

House size (sq ft).

2. Weighted Sum

$z = w \cdot \text{Size} + b$ (here $w = 0.25$, $b = -50$)

For size = 1200: $z = 0.25 \times 1200 - 50 = 250$

3. Apply ReLU

If $z > 0$, $\text{output} = z$. If $z \leq 0$, $\text{output} = 0$.

For size = 400: $z = 0.25 \times 400 - 50 = 50 \rightarrow \text{output} = 50$

For a tiny size = 100: $z = 0.25 \times 100 - 50 = -25 \rightarrow \text{output} = 0$

Why ReLU helps

- Keeps positive values — large houses give meaningful, positive predictions.
- Kills negatives — invalid or tiny inputs get zeroed out, so they don't mislead the model.
- Fast to train — simple function, avoids the vanishing-gradient problem seen in sigmoid/tanh.

Takeaway: ReLU is ideal for hidden layers in deep networks — it passes forward useful positive signals and blocks meaningless negatives. Here, only realistic house sizes end up contributing to learning.

2. Sigmoid

A dataset never stores a “sigmoid output” column — it only has raw **features** and true **labels**. Sigmoid is computed by the model during training, on top of those features.

Example Dataset: Student Pass / Fail

Hours Studied	Label (y)
1	0 (Fail)
3	0 (Fail)
5	1 (Pass)
7	1 (Pass)

Notice: the dataset only has **Hours Studied** (feature) and **Pass/Fail** (label). There is no sigmoid column here.

How Sigmoid Comes In (during training)

1. Weighted Sum

$z = w \cdot x + b$ — for 5 hours studied, $z = 1.2$

2. Sigmoid Activation

$$f(z) = 1 / (1 + e^{-z}) = 0.77$$

This is the model's predicted **probability of passing**.

3. Interpretation

If output $\geq 0.5 \rightarrow$ predict **Pass (1)**.

If output $< 0.5 \rightarrow$ predict **Fail (0)**.

Key point

- Dataset: only raw inputs (features) and true outputs (labels).
- Sigmoid: used inside the model to turn weighted sums into probabilities, during training/prediction.
- The sigmoid output itself is never stored in the dataset — the model computes it on the fly.

Takeaway: The dataset gives raw inputs and true labels. Sigmoid is applied during forward propagation to produce probabilities, which are compared against the labels to compute error and update weights.

3. Tanh (Hyperbolic Tangent)

Formula: $f(z) = (e^z - e^{-z}) / (e^z + e^{-z})$

- Range: (-1, 1)
- Zero-centered — negative inputs push toward -1, positive inputs push toward +1.
- Useful when you need to capture both positive and negative signals.

Example Dataset: Sentiment Analysis

Word	Weighted Sum (z)	Tanh Output	Interpretation
"Great"	2.0	0.96	Strongly positive
"Okay"	0.5	0.46	Mildly positive
"Bad"	-2.0	-0.96	Strongly negative
"Average"	-0.5	-0.46	Mildly negative

Step-by-Step (for the word "Great")

1. Weighted Sum

Suppose $w = 1$, $b = 0$. For "Great": $z = 2.0$

2. Tanh Activation

$$f(z) = (e^2 - e^{-2}) / (e^2 + e^{-2}) \approx 0.96$$

3. Interpretation

Output = 0.96 $\rightarrow$ strongly positive sentiment.

Why Tanh

- Zero-centered outputs (unlike sigmoid, which is always positive).
- Good for hidden layers when signals can be both positive and negative.
- Commonly used in RNNs for text and sequence data.

4. Softmax

Formula: $f(z_j) = e^{z_j} / \sum_j e^{z_j}$
Converts raw scores (logits) into a probability distribution across multiple classes.

Example Dataset: Cat vs Dog vs Bird

Image ID	Content	Label
Img_01	Cat photo	Cat
Img_02	Dog photo	Dog
Img_03	Bird photo	Bird

Step-by-Step (for Img_02, a Dog photo)

- 1. Input**
Model is given Img_02. True label (from dataset) = Dog.
- 2. Feature Extraction**
Network learns patterns: Cat → whiskers, round face. Dog → floppy ears, snout. Bird → beak, wings.
Img_02's features match dog-like patterns.
- 3. Weighted Sum (Logits)**
 $z = [\text{Cat: } 1.0, \text{ Dog: } 3.0, \text{ Bird: } 0.5]$
Dog gets the highest raw score.
- 4. Softmax**
Converts logits to probabilities: [Cat: 0.09, Dog: 0.84, Bird: 0.07]
- 5. Prediction**
Highest probability = 0.84 → Predicted class = **Dog**.
Compare with true label (Dog) → Prediction is correct.

```
Input Image (Dog)
--> Extract Features (ears, snout, fur)
--> Weighted Sum [1.0, 3.0, 0.5]
--> Softmax [0.09, 0.84, 0.07]
--> Prediction = Dog
--> Compare with Label = Dog -> Correct
```

Takeaway: Dog scored higher because its features matched dog-like patterns the model learned during training. Softmax then scales those raw scores into probabilities, and the highest one becomes the final prediction.

Quick Comparison

Function	Output Range	Best Used For
Sigmoid	(0, 1)	Binary classification (probabilities)
ReLU	[0, ∞)	Hidden layers in deep nets (fast training)
Tanh	(-1, 1)	Hidden layers with +/- signals (sentiment, sequences)
Softmax	(0, 1), sums to 1	Multi-class classification